Two-Stage Product Recommendation for the Olist Marketplace

Capstone: Post Graduate Diploma in AI & Machine Learning

Daniel Jethro Monzada · July 2026

~100k real Brazilian eCommerce orders (2016-2018) · candidate generation + supervised re-ranking · fairness-audited

Repo: `github.com/jmonzada/ecommerce-recommender-capstone`

<small>Data: Brazilian E-Commerce Public Dataset by Olist (Kaggle, CC BY-NC-SA 4.0)</small>

One number, and the shape of the answer

96.9% of Olist's 96,096 customers have placed exactly one order.

- Business question: can a recommender pull customers into a **second purchase**, and widen exposure beyond bestsellers?
- Seller angle: Olist's pitch to small merchants is *exposure*, and a popularity-only feed quietly breaks it.

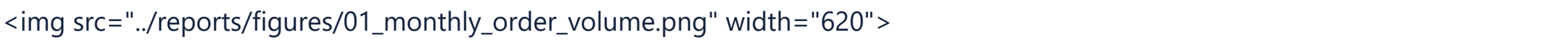
Architecture: two stages, like production systems

1. **Candidate generation**: item-item CF, content similarity, truncated SVD, popularity, hybrid blend
2. **Supervised ranker**: LR / RF / XGBoost scoring "will this (customer, product) pair convert?"

The ranker is feature-based, which is where SHAP, PDP/ICE and the fairness audit genuinely work.

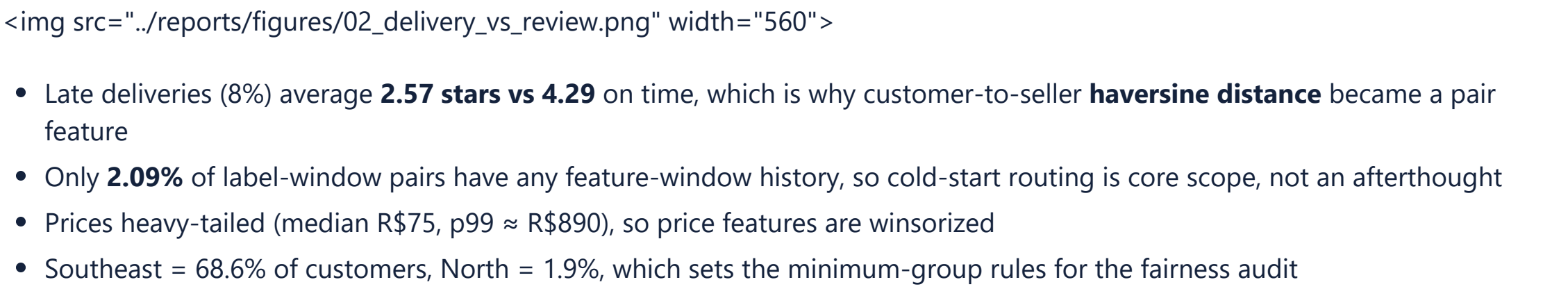
*Where it lands: on a temporally clean holdout the full pipeline beats shortlist-only by **+20%** (CI excludes zero); see slide 7.*

Evaluation protocol (the load-bearing part)



- **Three temporal windows** (`configs/windows.yaml`): features 2017-01 to 2018-02 · labels 2018-03 to 2018-05 · holdout 2018-06 to 2018-08 (post-Aug tail collapses, dropped)
- **Every aggregate** (popularity, similarities, SVD factors, RFM…) computed on the feature window only; audit cells assert max timestamps, including review *creation* dates (caught 6.7% leakage)
- Candidate gen: **leave-last-order-out** on 2,789 repeat buyers (840 tuning / 1,949 evaluated), bootstrap CIs, K = 10/50/100 + MRR
- Ranker: **GroupKFold by customer**; negatives 1:4, half hard (from candidate lists), half random; random-only negatives make the task popularity-easy and mismatch serving (the Step 3 random-negative screen already sat at AUC ≈ 0.81 with weaker features)

EDA that shaped the design



- Late deliveries (8%) average **2.57 stars vs 4.29** on time, which is why customer-to-seller **haversine distance** became a pair feature
- Only **2.09%** of label-window pairs have any feature-window history, so cold-start routing is core scope, not an afterthought
- Prices heavy-tailed (median R$75, p99 ≈ R$890), so price features are winsorized
- Southeast = 68.6% of customers, North = 1.9%, which sets the minimum-group rules for the fairness audit

Stage 1: candidate generation (repeat buyers, leave-last-order-out)

Model	Hit@10	95% CI	MRR	Coverage	Long-tail
Popularity	0.031	[0.023, 0.038]	0.013	0.0003	0.00
Item-item CF	0.055	[0.045, 0.065]	0.037	0.042	0.83
Content-based	0.171	[0.154, 0.187]	0.135	**0.377**	0.79
SVD (k=256)	0.054	[0.045, 0.065]	0.046	0.019	0.03
Hybrid (w=0.25)	**0.203**	[0.184, 0.221]	0.138	0.371	0.76

- Paired per-user difference, hybrid minus content: **+0.032 [+0.022, +0.042]**, a real winner, not noise
- Honesty note: **62-69% of hits are repurchases**; CF is signal-starved (co-purchase matrix has 8,710 pairs)
- Popularity's coverage is 10 products. That's the baseline the marketplace ships without ML.
- Caveat: LOO trades temporal strictness for coverage; decision-grade numbers are the end-to-end table (slide 7)

Stage 2: conversion rankers (sampled negatives give relative numbers)

Ranker	CV AUC	Holdout AUC	PR-AUC	F1@τ
Logistic regression	0.697	0.785	0.414	0.475
Random forest	0.852	0.865	0.619	0.569
XGBoost	0.853	0.864	0.614	**0.578**

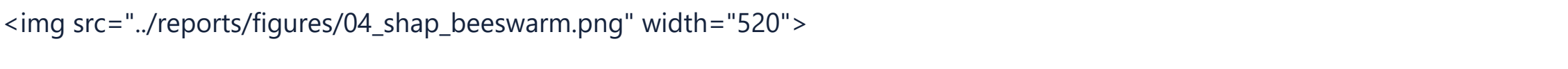
- Tuned via randomized search, `GroupKFold` by customer; τ picked on **out-of-fold** predictions
- RF train AUC 0.945 vs holdout 0.865; the overfit gap LR doesn't have; XGBoost wins on F1 with a smaller gap
- These metrics **compare rankers against each other**; sampled 1:4 negatives make absolute values meaningless

The headline table: end-to-end on 18,390 holdout customers

Pipeline	Hit@10	95% CI	NDCG@10
Popularity only	0.0114	[0.0099, 0.0130]	0.0052
Stage 1 only (routed hybrid)	0.0122	[0.0105, 0.0139]	0.0062
Two-stage + LR	0.0061	[0.0051, 0.0073]	0.0032
Two-stage + RF	0.0132	[0.0115, 0.0150]	0.0064
Two-stage + XGBoost	**0.0146**	[0.0128, 0.0165]	0.0069

- Paired per-customer difference vs Stage 1: **+0.0024 [+0.0008, +0.0041]**, i.e. **+20% relative**, CIs exclude zero
- Re-ranking is not free: the LR re-rank *halves* performance; stage 2 must earn its place
- Routing reality: **98.5% of holdout customers are cold**, served regional/global popularity; warm cohort (n=277) is underpowered and slightly prefers raw hybrid, flagged as future work, not hidden

Explainability: SHAP on the ranker



- Top signals: product popularity, co-purchase signal, price/spend match, seller stats, distance
- **The counterintuitive one:** within candidate lists, high popularity gets a *negative* gradient; hard-negative training taught the ranker "popular but still not bought" (PDP peaks then collapses; LIME agrees on signs)
- Cold-start waterfall: with no customer history, product & seller features carry the score, exactly what you'd want
- This SHAP-to-plain-English pipeline later feeds the LLM explanations (Step 9)

Fairness audit: region as socioeconomic proxy (IBGE-cited)

Evaluation population = **Stage-1 candidate lists** (919,500 pairs); per-user random negatives would fake parity. Groups under n<30 positives suppressed (North).

Region	TPR@τ	AUC
Northeast	**0.222** [0.139, 0.319]	**0.825**
Center-West	0.100	0.679
South	0.031	0.709
Southeast	0.028	0.714

- **TPR gap 0.194, favoring the Northeast**, and it persists in final lists (NE hit@10 0.031 vs 0.010-0.014)
- Mechanism found: Stage-1 **retrieval recall** is already higher for NE (0.043 vs 0.025-0.032); the disparity starts at candidate generation, not the ranker
- Fairness-through-unawareness buys nothing: dropping geography features costs accuracy (AUC 0.723 to 0.713) while gaps barely move; the τ-fixed TPR-gap widening (0.194 to 0.292) is partly a calibration artifact; freight & price encode geography anyway

Provider-side bias & mitigations (matched before/after)

Exposure Gini **0.9986**; small sellers = 5.9% of catalog, **0.02% of top-10 slots**.

Mitigation	Result
ThresholdOptimizer (region)	Honest negative, didn't transfer to holdout
Score-penalty re-rank (λ sweep)	Inert; bias sits at *candidate generation*
Candidate-stage exploration	**Backfired: -73% hits** (the anti-popularity gradient punishes injected popular-ish items)
Reserved-slot injection (3 of 10)	**-14.5% Hit@10** for coverage 0.071 to 0.102, long-tail ×2.1, small sellers ×4.3

South pays the most: paired Δ -0.0045 [-0.0078, -0.0021], reported and gated on fresh-window validation before serving.

From notebooks to a running system

- **Serving:** FastAPI `GET /recommend/{customer_id}` runs the *exact evaluated pipeline* (`src/pipeline.py` shared with the audit notebook); cold-start routing built in; Docker image bakes data + artifacts
- **MLOps:** configs in YAML, MLflow on every training run, CI (ruff + 38 tests) green since first push, monitoring plan (routing-share drift, then score PSI, then online hit-rate, then monthly exposure), rollback = previous git tag
- **GenAI (Step 9):** each recommendation's TreeSHAP signals become value-aware plain-English glosses, then one grounded Claude sentence, cached & committed; the grounding bug we caught (asserting a region for an unknown visitor) is preserved as evidence
- Demo video: `docs/media/demo.mp4` (41s, all three routes + explanations)

Limitations & what I'd do next

Limitations (also in the report, not fine print):

- All numbers are **offline proxies**; validating uplift needs an A/B test
- Warm-cohort re-rank question is underpowered (n=277); route-conditional re-ranking is the obvious experiment
- Repurchase drives most personalization hits; CF is starved at 1.04 products/order
- Fairness mitigation trade-off selected on the audit window, so **fresh-window validation before enabling**

Next: online experiment on repeat buyers · route-conditional re-rank · richer content features (the dataset has no text/images) · category-level objectives for cross-sell

Everything reproducible: seeded runs, pinned configs, committed artifacts, one-command serving.